Planilla de Corrección Proyecto 1 ***Comisión 11***

Nota final: C	**Aprobado**

Análisis preliminar del proyecto: *descarga y compilación del código fuente; uso de la aplicación en ejecución.*	
¿Todos los alumnos mostraron actividad en el uso del repositorio y las defensas?	**SI** - Sin observaciones.
¿El código se compila?	**SI** - Sin observaciones.
¿La aplicación permite fácilmente seleccionar cualquiera de los tres modos de juego y los dos dominios de aplicación?	**SI** - Sin observaciones.
El juego respeta todos los puntos enunciados y requeridos?	**SI** - Observaciones: - Las imágenes de obstáculos y el fondo no varían entre el dominio original y el dominio alternativo. Sin embargo, el patrón factory está bien implementado y hay una carpeta para cada dominio, por lo que cambiando las imágenes debería funcionar. - En ocasiones, el Troll Amarillo amaga a saltar pero no realiza el salto.
¿El juego se comporta como se espera (estado inicial, condición de finalización, transición de niveles y reinicio en caso de perder)?	**SI** - Observaciones: - En ocasiones al iniciar una nueva partida luego de haber perdido contra calabazas y fantasmas, quedan imágenes de estos enemigos. (Solo la imágen, lógicamente no existen porque no interactúan con el jugador ni se mueven)
¿El juego se comporta como se espera ante la interacción con los power-ups (cambio de estado, cambio de sprites)?	**SI** - Sin observaciones
¿El juego se comporta como se espera ante la interacción con los enemigos (condición para muerte, saltos sobre elementos esquivables mediante esta modalidad)?	**SI** - Sin Observaciones
¿El juego se comporta como se espera respecto del lanzamiento de las bolas de nieve y el	**SI** - Observaciones: - Los disparos de bola de nieve tienen caída como en el juego original.

congelamiento del enemigo?	
¿El juego opera el ranking de forma consistente?	**SI** - Sin observaciones
Virtudes de la GUI y usabilidad	**SI** - Observaciones: - La transición de piso tiene una animación en la cual los elementos del piso actual van desapareciendo y los del piso siguiente van apareciendo. - Kamakichi lanza enemigos bomba como en el juego original. - El Bro puede atravesar plataformas desde abajo hacia arriba para subir como en el juego original.
Debilidades de la GUI y usabilidad	**SI** - Observaciones: - Los elementos del juego son pequeños. - Varias imágenes en el dominio alternativo son las mismas que las del dominio original
Errores y/o características que deberían modificarse	**NO** -Sin Observaciones

(sigue debajo)

Análisis preliminar del proyecto: *diagramas reducidos, extendidos y de secuencia.*	
¿Se encuentran accesibles los *diagramas reducidos* y *extendidos* de clases de la entrega final?	**SI** - Observaciones: - Son los archivos: Diagrama-extendido-puntuable-3 y Diagrama-reducido-puntuable-3
Análisis, elogios, críticas y sugerencias del diagrama de clases reducido respecto de las decisiones de modelado tomadas.	**SI** - Observaciones: Reducido: - El diagrama reducido está muy incompleto. Faltan la gran mayoría de relaciones y asociaciones que sí se encuentran en Diagrama-reducido-puntuable-2. Además se utilizan elementos en el diagrama (círculos) que no pertenecen a UML. Extendido: - SnowBro tiene atributos relacionados a la duración de los efectos y a la duración de las animaciones. Estas son características que no son del Bro, deberían estar en otra clase, para cumplir el principio de responsabilidad única. La duración de las animaciones está más relacionada con las Skins. - En EstadoMovimientoBro tiene métodos estáEnModoEscalera() y estáSubiendoEscalera y SnowBro que en realidad debería modelarse como otro Estado. - La duración de las pociones y la cantidad de estadística que otorgan son características del Efecto de la Poción, no características del Bro. - Los atributos enteros en los Enemigos de los niveles de congelación y su duración de un enemigo son características de un EstadoSemicongelado, en lugar de tener un estado para cada nivel de congelación. Estos estados solo tienen un método, lo cual indica que no se delegó el comportamiento de las interacciones según el estado a estas clases (movimientos y colisiones) - Sucede lo mismo con los estados de los EnemigosBomba - El patrón visitor debería haberse diseñado con un método para cada clase concreta para poder tener mayor flexibilidad a la hora de implementar la interacción entre dos elementos del juego y no necesitar métodos tales como: esSueloSolido() y destruyeBolaDeNieve(). - En EnemigoVoladorDiagonal, los atributos TECHO_MUNDO y PISO_MUNDO son características del Nivel no del Estado.

(sigue debajo)

Análisis del código fuente: *inspección del código fuente entregado.*	
¿El código fuente se encuentra adecuadamente organizado en paquetes?	**SI** - Sin observaciones.
¿El código exhibe de forma consistente y adecuada las prácticas propuestas por Clean Code?	**NO** - Sin observaciones. - Observaciones: - Se observan métodos de movimiento en los Enemigos y en EstadosMovimiento, algunos métodos que arman la GUI y los métodos de verificación de colisiones son muy extensos. Deberían subdividirse en métodos auxiliares para mejorar la legibilidad - Se observa nombres de variables de una letra (que no son “x” o “y”) - Podrían utilizarse constantes para los números que indican que sprite deben tener los personajes en determinado momento. - Se respetan nombres significativos
¿Existe una adecuada división de responsabilidades entre las clases que implementan cuestiones de lógica/gráfica?	**SI**. - Observaciones: Utilizan el patrón observer para actualizar la gráfica En EnemigoKamachiVertical se utiliza una constante gráfica para determinar el medio del nivel. Esto no es correcto, el medio del nivel debería determinarse lógicamente.
¿Se implementa de forma consistente la <u>detección</u> de colisiones?	**SI** - Sin observaciones.
¿Se implementa de forma consistente la <u>resolución</u> de colisiones?	**SI** - Observaciones: Se utiliza el patrón visitor correctamente con métodos de clases generales. Sin embargo esto provocó que tuviesen que agregar métodos extra para implementar interacciones entre algunos pares de elementos que podrían haberse evitado si se utilizaba el patrón visitor con métodos para todas las clases concretas.
Virtudes	**SI** - Observaciones: La comisión dejó por escrito las pautas de clean code que siguieron para el desarrollo del proyecto. Este tipo de práctica es muy positiva para integrar rápidamente a un integrante nuevo del equipo al proceso de desarrollo.
Debilidades y/u otros errores a corregir	**SI** - Observaciones (Debilidades): No se implementa correctamente el patrón state para la congelación. Se le asignan números a los estados y según los valores, el Enemigo ejecuta la lógica. Esto es poco extensible y provoca que los métodos de movimiento en

	los Enemigos queden extensos. Se debería delegar la lógica de interacciones y movimientos a las clases Estado. En casos como crearPowerUp de RanaDeFuego hay cadena de getters para generar el PowerUp. Esto reduce el encapsulamiento entre distintas partes del sistema. La Rana debería tener una referencia a la fábrica para crear el PowerUp.